

# ORGNZ-08: Grail Segments

---

**Series:** ORGNZ | **Notebook:** 8 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

## Overview

---

**Grail segments** are logical groupings of data that enable real-time filtering across massive datasets without creating thousands of individual rules. Segments bring business context to observability data and are consistently available across the Dynatrace platform.

## Prerequisites

---

| Requirement | Details |  
|---

---

## Table of Contents

---

1. [Segments vs Buckets](#)
2. [Segment Use Cases](#)
3. [Segment Structure](#)
4. [Segment Limits](#)
5. [Segment Design Patterns](#)
6. [Leveraging OneAgent Enrichments](#)
7. [Entity Relationship Strategies](#)
8. [Testing Segments](#)
9. [Segments and Access Control](#)
10. [Segment Design Checklist](#)

---

-----|-----|

| **Dynatrace Environment** | SaaS environment with Grail enabled |

| **Permissions** | `segment:read` for viewing, `segment:write` for creating |

| **Knowledge** | Completed ORGNZ-01 through ORGNZ-07 |

## Learning Objectives

---

By the end of this notebook, you will:

- Understand segments vs buckets
- Design effective segment rules
- Use dynamic variables in segments
- Apply entity relationships for comprehensive filtering

## Segments vs Buckets

---

| Aspect                | Buckets                    | Segments                   |
|-----------------------|----------------------------|----------------------------|
| <b>Purpose</b>        | Physical data storage      | Logical data filtering     |
| <b>Retention</b>      | Controls data retention    | No impact on retention     |
| <b>Data types</b>     | One type per bucket        | Multiple types per segment |
| <b>Access control</b> | Can be used with IAM       | Data filtering only        |
| <b>Cost impact</b>    | Direct storage costs       | No direct cost impact      |
| <b>Query scope</b>    | Storage-level partitioning | Query-time filtering       |

**Key insight:** Buckets store data; segments filter it. Use buckets for retention and cost allocation, segments for dynamic data views.

# Segment Use Cases

---

## 1. Team-Based Views

Segment: "Platform Team"

Includes: Infrastructure hosts, Kubernetes clusters, platform services

Benefit: Team sees only relevant data, faster troubleshooting

## 2. Application Isolation

Segment: "Checkout Application"

Includes: Checkout service, related databases, frontend components

Benefit: Complete application view across logs, traces, metrics

## 3. Environment Filtering

Segment: "Production Only"

Includes: Entities with environment=production tag

Benefit: Focused production monitoring without dev/test noise

## Segment Structure

---

```
Segment:
  name: "segment-identifier"
  displayName: "Human Readable Name"
  description: "What this segment filters"

  includes:
    - type: logs
      filter: "host.name starts-with 'prod-'"

    - type: dt.entity.host
      filter: "tags contains 'environment:production'"

    - type: spans
      filter: "service.name == 'checkout'"

  variables:
    - name: environment
      values: ["production", "staging"]
```

## Segment Limits

---

| Limit                                | Value | Notes                                    |
|--------------------------------------|-------|------------------------------------------|
| Maximum includes (rules) per segment | 20    | Plan rule complexity accordingly         |
| Include blocks per data/entity type  | 1     | Cannot have multiple rules for same type |
| Expressions per filter condition     | 10    | Maximum conditions per filter            |
| Values per variable                  | 2,000 | Maximum variable values                  |

## Wildcard Restrictions

| Pattern     | Supported? | Notes                                      |
|-------------|------------|--------------------------------------------|
| starts-with | Yes        | Must have at least one preceding character |
| contains    | No         | Not supported                              |
| ends-with   | No         | Not supported                              |

## Segment Design Patterns

---

### Pattern 1: Team-Based Segment

```
name: team-platform
displayName: "Platform Team"
description: "All infrastructure and platform services"

includes:
  - type: dt.entity.host
    filter: "tags contains 'team:platform'"

  - type: logs
    filter: "host.group starts-with 'platform-'"

  - type: spans
    filter: "service.name starts-with 'platform-'"

```

## Pattern 2: Application-Based Segment

```
name: app-checkout
displayName: "Checkout Application"
description: "Checkout service and dependencies"

includes:
  - type: dt.entity.service
    filter: "tags contains 'app:checkout'"

  - type: logs
    filter: "service.name contains 'checkout'"

  - type: spans
    filter: "service.name contains 'checkout'"
```

## Pattern 3: Environment-Based Segment

```
name: env-production
displayName: "Production Environment"
description: "Production workloads only"

variables:
  - name: env
    values: ["production", "prod"]

includes:
  - type: dt.entity.host
    filter: "tags contains 'environment:${env}'"

  - type: logs
    filter: "host.group starts-with 'prod-'"
```

## Leveraging OneAgent Enrichments

---

OneAgent automatically enriches signals with metadata for precise filtering:

| Enrichment                      | Source                   | Example Use             |
|---------------------------------|--------------------------|-------------------------|
| <code>host.group</code>         | Host group configuration | Filter by host group    |
| <code>k8s.namespace.name</code> | Kubernetes namespace     | Filter by namespace     |
| <code>k8s.cluster.name</code>   | Kubernetes cluster       | Filter by cluster       |
| <code>cloud.provider</code>     | Cloud detection          | Filter by AWS/Azure/GCP |
| <code>cloud.region</code>       | Cloud region             | Filter by region        |
| <code>tags</code>               | Entity tags              | Filter by custom tags   |

## Host Group Filtering Example

```
includes:
  - type: logs
    filter: "host.group == 'prod-web-tier'"

  - type: dt.entity.host
    filter: "hostGroup == 'prod-web-tier'"
```

## Entity Relationship Strategies

### Top-Down (Host-Centric)

Start from hosts and include related entities:

```
includes:
  - type: dt.entity.host
    filter: "tags contains 'team:platform'"

  - type: dt.entity.process_group
    relationship: "runsOn dt.entity.host"

  - type: dt.entity.service
    relationship: "runsOnProcessGroup dt.entity.process_group"
```

## Bottom-Up (Service-Centric)

Start from services and include supporting infrastructure:

```
includes:
  - type: dt.entity.service
    filter: "tags contains 'app:checkout'"

  - type: dt.entity.process_group
    relationship: "hostsService dt.entity.service"

  - type: dt.entity.host
    relationship: "runsProcessGroup dt.entity.process_group"
```

## Testing Segments

---

```
// Test host group filtering (simulating segment rule)
fetch dt.entity.host
| filter hostGroup starts-with "prod-"
| fields entity.name, hostGroup, tags
| limit 20
```

```
// Test tag-based filtering
fetch dt.entity.service
| expand tags
| filter contains(tags, "team:platform")
| fields entity.name, tags
| limit 20
```

```
// Test log filtering by host group
fetch logs
| filter startsWith(host.group,"prod-")
| summarize count = count(), by:{host.group}
| sort count desc
| limit 10
```

## Segments and Access Control

---

**Important:** Segments are for **data filtering only**, not access control.

| Function                    | Segments | IAM Policies |
|-----------------------------|----------|--------------|
| Filter data in queries      | Yes      | No           |
| Restrict user access        | No       | Yes          |
| Enforce security boundaries | No       | Yes          |
| Dynamic data views          | Yes      | No           |

Segments are governed by existing access controls - users only see data they're authorized to access.

## Segment Design Checklist

---

- ☐ Defined clear segment purpose
- ☐ Identified all data types to include
- ☐ Verified metadata consistency (tags, host groups)
- ☐ Planned rule structure (max 20 includes)
- ☐ Considered entity relationships
- ☐ Tested filter conditions with DQL
- ☐ Documented segment usage
- ☐ Planned variable usage for flexibility

## Next Steps

---

Continue with the ORGNZ series:

- **ORGNZ-09**: Enterprise Patterns

## References

---

- [Grail Segments](#)
  - [Entity Model](#)
  - [Segments blog post](#)
-

*This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*